



NVIDIA FP8

Sparsh Mittal

Understanding Scaling Factor

Example 1

Assume we have 3 numbers: $1.1 \cdot 10^3$, $2.9 \cdot 10^3$, $8.4 \cdot 10^3$

How to store these efficiently?

Answer: store 10^3 as the single scaling factor for all the numbers and the rest of the number separately. Thus, store:

$[1.1, 2.9, 8.4]$ and 10^3

Example 2

Assume we have 6 numbers: $1.1 \cdot 10^3$, $2.9 \cdot 10^3$, $8.4 \cdot 10^3$, $9 \cdot 10^4$, $6.7 \cdot 10^4$, $3.1 \cdot 10^4$

How to store these efficiently?

Answer: store two scaling factors: 10^3 and 10^4 for two sets of numbers and store the rest of the number separately. Thus, store:

[1.1, 2.9, 8.4] and 10^3

[9, 6.7, 3.1] and 10^4

Example 3

Assume we have 6 numbers: $1.1 \cdot 10^9$, $2.9 \cdot 10^9$, $8.4 \cdot 10^9$, $9 \cdot 10^{11}$, $6.7 \cdot 10^{11}$, $3.1 \cdot 10^{11}$

How to store these efficiently?

Answer: store one global scaling factor of 10^{10} . Then, store two group-scaling factors: 10^1 and 10^{-1} . Store rest of the number separately. Thus, store:

$\left. \begin{array}{l} [1.1, 2.9, 8.4] \text{ and } 10^1 \\ [9, 6.7, 3.1] \text{ and } 10^{-1} \end{array} \right\} 10^{10}$

Example 4

Assume we have 3 numbers: 665, 855 and 1045.

Trying to use a scaling factor of 500 or 1000 will lead to lot of error.

It is much better to use a scaling factor of 95, and then, the numbers are [7,9, 11].

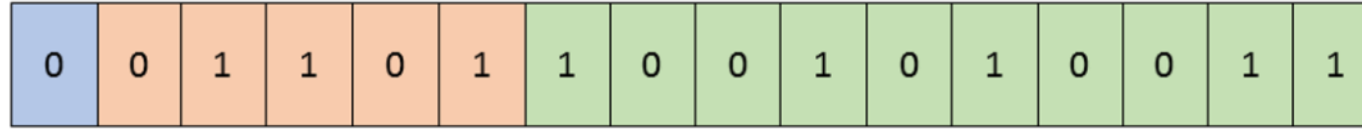
Lesson: power-of-ten scaling factors are not always good.

Understanding FP8 Format and Scaling Factor

FP32 = 0.3952

sign exponent mantissa

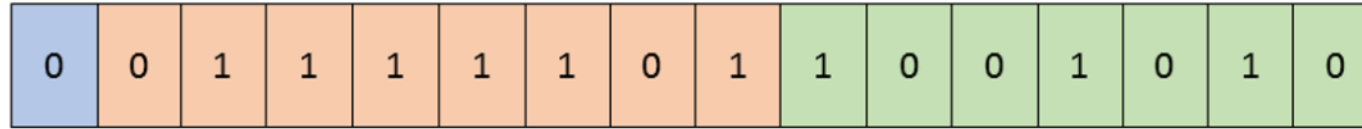
FP16



= 0.395264

More precision

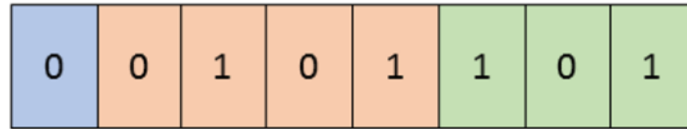
BF16



= 0.394531

More range

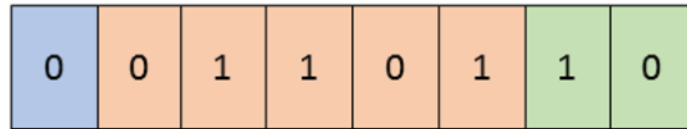
FP8 E4M3



= 0.40625

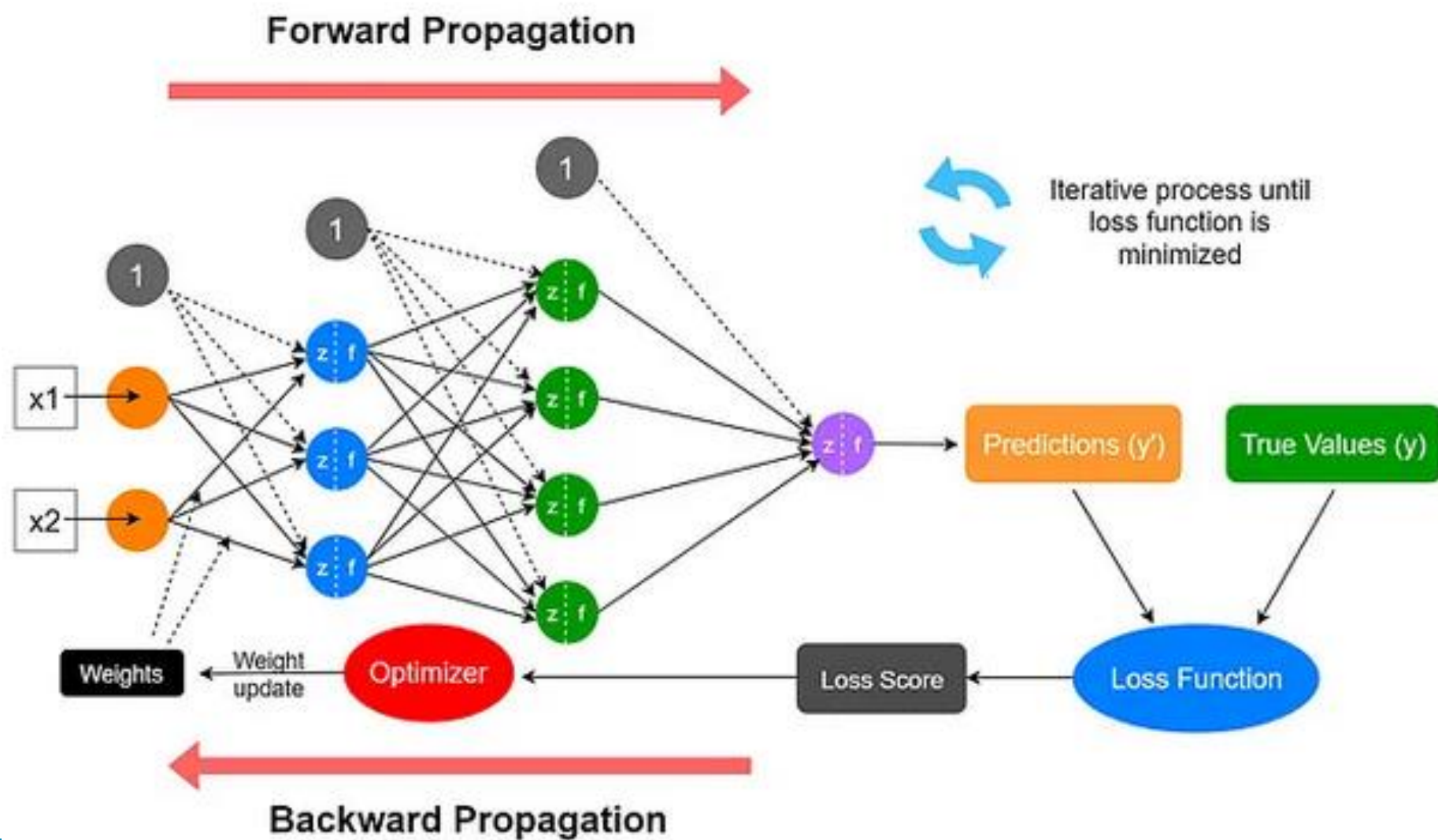
More precision

FP8 E5M2

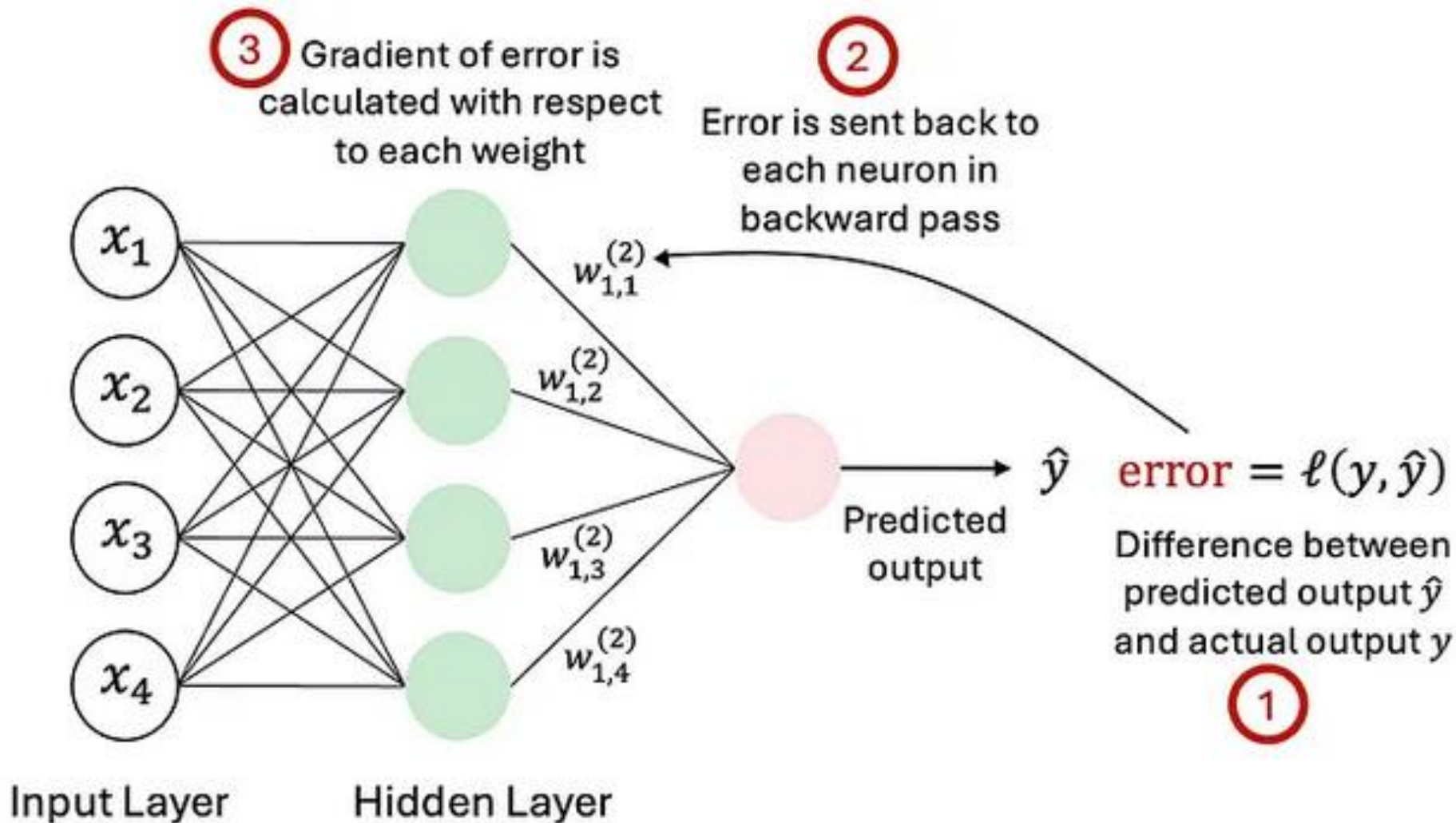


= 0.375

More range



Backward pass



FP8 splits into two variants:

E4M3 prioritizes precision for **forward passes**, where weights and activations benefit from finer-grained values.

Its approximate range of ± 448 , along with the ability to represent NaN, accommodates most layer outputs without overflow.

E5M2 trades mantissa bits for a wider dynamic range ($\pm 57,344$, $\pm \text{inf}$, nan).

This broader range is crucial for **backward passes**, where gradients can vary significantly in magnitude.

In the backward pass, gradients flowing through the network typically are less susceptible to the loss of precision, but require higher dynamic range.

BF16 vs FP8

BF16's 8 exponent and 7 mantissa bits offer a vast dynamic range (from $1e-38$ to $1e38$), which enables it to represent the distributions of weights, activations, and gradients without scaling factors.

FP8's double datatype (E4M3: with a range up to approximately ± 448 , and E5M2: ± 57344) coupled with scaling factors, enables more efficient hardware utilization compared to BF16 without sacrificing convergence.

FP8 vs MXFP8

The main difference between “regular” FP8 and MXFP8 lies in the granularity of the scaling.

In FP8, **each tensor** has a single FP32 scaling factor, so all values in the tensor need to “fit” within the dynamic range of the FP8 datatype.

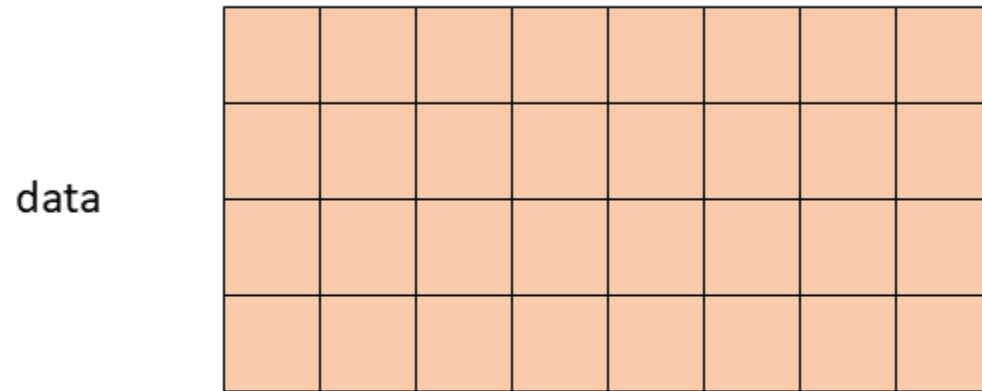
This requires using the less precise E5M2 format to represent some tensors in the network (like gradients).

MXFP8 addresses this by assigning a different scaling factor to **each block** of 32 consecutive values.

This allows all values to be represented with the E4M3 datatype.

MXFP8 and block scaling

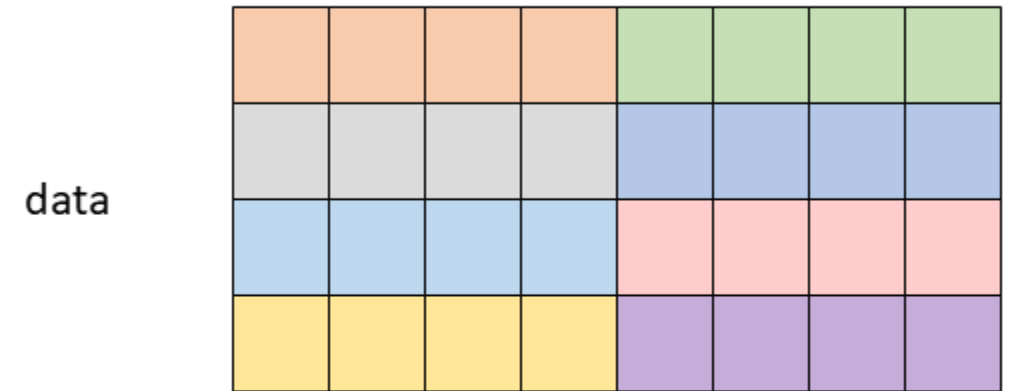
FP8



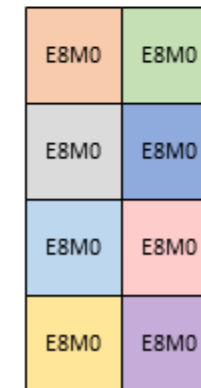
Scaling
factor



MXFP8



Scaling
factor



- MXFP8 uses multiple scaling factors for a single tensor.
- The picture shows only 4 values per block for simplicity, but real MXFP8 has 32 values per block.

FP8



Due to multiple scaling factors, tensor's dynamic range requirements are reduced and so E4M3 format can be used as far fewer elements get saturated to 0.

MXFP8



after
scaling

